第 8 章　数字系统设计基础

8.1　数字系统基本概念

随着现代电子技术的快速发展，数字系统已经广泛应用于计算机、通信、自动控制、智能电网、工业控制和人工智能等领域。从简单的电子时钟，到复杂的计算机处理器、智能机器人和嵌入式控制系统，本质上都属于数字系统。

所谓数字系统，是指以数字信号作为信息载体，利用数字逻辑电路对信息进行存储、运算、传输和控制的电子系统。数字系统中的信息通常采用二进制形式表示，即用“0”和“1”两种逻辑状态表示各种数据和控制信号。

由于数字信号抗干扰能力强、精度高、易于存储和处理，因此数字系统在现代电子技术中占据核心地位。在现代数字系统设计中，传统以门电路连接为核心的设计方式已经逐渐向硬件描述语言（Hardware Description Language，HDL）设计方式转变。设计者不再直接依赖物理连线完成逻辑结构搭建，而是通过抽象化描述数字系统的功能、结构与行为，再借助 EDA 工具完成综合、仿真与实现。这种设计方式不仅提高了设计效率，也使复杂系统的模块化开发成为可能。

8.1.1　数字系统的组成

从功能角度来看，数字系统通常由以下几个部分构成：

1. 数据通路（Data Path）

数据通路负责完成数据的传输、运算与存储，包括加法器、寄存器、移位器、计数器以及运算逻辑单元等模块。数据通路是数字系统中实现数据处理的核心部分。

2. 控制单元（Control Unit）

控制单元用于协调系统内部各模块的工作过程，通过产生控制信号决定数据流向和状态转换。控制单元通常采用有限状态机结构实现。

3. 存储单元（Memory Unit）

存储单元用于保存程序、数据以及系统状态，包括寄存器、RAM、ROM 等器件。随着可编程逻辑器件的发展，片上存储资源已经成为数字系统的重要组成部分。

4. 输入输出接口（I/O Interface）

输入输出接口实现数字系统与外部设备之间的数据交互，包括按键输入、显示驱动、串口通信以及传感器接口等。

5. 时钟与复位模块

同步数字系统通常依赖统一时钟实现同步控制。时钟信号决定系统运行节奏，而复位信号用于初始化系统状态，确保系统稳定运行。

从结构上看，一个完整的数字系统实际上是由多个模块按照一定层次关系组合形成的。现代数字系统设计强调模块化与层次化思想，通过自顶向下的方法逐步完成系统构建。

8.1.2 数字系统的特点

数字系统具有以下几个典型特点：

（1）离散性

数字系统中的信号通常只有有限个状态，最常见的是二值逻辑“0”和“1”。这种离散表示方式提高了系统抗噪声能力。

（2）逻辑性

数字系统通过逻辑运算实现功能控制，其本质是逻辑关系的组合与状态变化。

（3）同步性

多数数字系统采用同步时钟控制，各模块在统一时钟作用下协调工作。同步设计有利于提高系统稳定性与可预测性。

（4）层次化

复杂数字系统通常采用层次化结构设计，通过模块划分降低系统复杂度。

（5）可编程性

随着 FPGA 与 CPLD 等可编程逻辑器件的发展，数字系统可以通过修改硬件描述代码实现功能重构，提高系统灵活性。

8.1.3 数字系统的分类

按照不同标准，数字系统可以进行多种分类。

1. 按照逻辑特性分类

（1）组合逻辑系统

组合逻辑系统的输出仅与当前输入有关，与历史状态无关。

（2）时序逻辑系统

时序逻辑系统的输出不仅与当前输入有关，还与系统原有状态有关，系统内部通常包含存储单元。

2. 按照时钟方式分类

（1）同步数字系统

同步系统中所有状态变化均在统一时钟控制下完成。

（2）异步数字系统

异步系统不依赖统一时钟，其状态变化由输入变化直接触发。

3. 按照实现方式分类

（1）固定逻辑系统

功能由硬件结构直接决定，设计完成后难以修改。

（2）可编程逻辑系统

系统功能可通过程序或硬件描述语言重新定义，具有较高灵活性。

8.1.4 数字系统的发展趋势

随着集成电路技术和 EDA 技术的发展，数字系统设计正在向以下方向演进：

1. 高集成度

现代 FPGA 器件已经集成大量逻辑资源、DSP 模块与片上存储器，可实现复杂系统级设计。

2. 高层次抽象

设计描述逐渐从门级转向 RTL 级、系统级甚至行为级，提高开发效率。

3. IP 核复用

通过标准化 IP 核复用，可显著缩短设计周期。

4. 软硬件协同设计

现代嵌入式系统越来越强调软件与硬件协同优化。

5. 智能化 EDA 设计

人工智能技术逐渐应用于综合优化、时序分析以及自动布局布线等环节。

8.1.5 硬件描述语言在数字系统设计中的作用

硬件描述语言是现代数字系统设计的重要基础。设计者可以通过 HDL 描述系统结构、行为与时序关系，再由 EDA 工具完成逻辑综合与硬件实现。

Verilog HDL 是一种广泛应用的硬件描述语言，具有语法简洁、层次清晰以及易于综合等特点。Verilog 既能够描述底层逻辑门连接，也能够描述复杂系统行为，因此在 FPGA 与 ASIC 设计中得到广泛应用。

硬件描述语言具有以下优势：

（1）提高设计抽象层次；

（2）支持模块化设计；

（3）便于仿真验证；

（4）提高系统可移植性；

（5）支持自动综合与优化。

因此，基于 Verilog HDL 的设计方法已经成为现代数字系统设计的主流方式。

8.2 数字系统设计的一般过程

数字系统设计是一个从功能需求到硬件实现的完整过程。对于简单数字电路，设计者可以直接根据逻辑表达式搭建电路；但对于复杂数字系统，仅依靠传统手工设计已经难以完成，因此需要采用规范化、工程化的设计流程。

数字系统设计通常从需求分析开始。设计人员首先需要明确系统要实现的功能，包括输入输出关系、运行方式、性能要求以及系统约束条件。

在明确功能需求后，需要进行系统方案设计。设计人员根据系统目标确定整体结构，并划分功能模块。例如，一个数字系统可能包括输入接口模块、数据处理模块、时钟模块、状态控制模块以及输出驱动模块等。模块化设计思想是现代数字系统设计的重要特征，它能够降低系统复杂度，提高设计效率。

完成总体方案设计后，需要进一步进行逻辑设计。对于组合逻辑部分，需要根据输入输出关系建立真值表、逻辑表达式或逻辑框图；对于时序逻辑部分，则需要建立状态图、状态表以及状态转移关系。在这一过程中，逻辑化简和状态优化十分重要，因为它们会直接影响系统硬件资源消耗和运行性能。

在传统设计方法中，设计人员通常利用逻辑门、触发器和中规模集成器件直接搭建电路。而在现代数字系统设计中，更常采用硬件描述语言进行功能描述。设计者通过 VHDL 或 Verilog 等硬件描述语言描述系统功能，再利用 EDA 软件完成逻辑综合和电路实现。

逻辑综合之后，需要对设计结果进行仿真验证。仿真是数字系统设计中非常重要的步骤，其目的是检查设计是否满足功能要求。仿真通常包括功能仿真和时序仿真。功能仿真主要验证逻辑功能是否正确；时序仿真则进一步考虑器件延时和时钟关系，验证系统运行是否可靠。

完成仿真验证后，需要进行硬件实现。对于基于可编程逻辑器件的设计，EDA 工具会自动生成下载文件，并将其配置到 FPGA 或 CPLD 中。下载完成后，还需要进行实际硬件测试，以验证系统在真实运行环境中的工作情况。

在实际工程中，数字系统设计往往不是一次完成的，而是一个不断修改和优化的过程。当测试结果不满足要求时，设计人员需要重新调整逻辑结构、优化时序关系，甚至重新划分系统模块。因此，数字系统设计具有明显的迭代特征。

现代数字系统设计强调“自顶向下”的设计思想。所谓“自顶向下”，是指先从系统整体功能出发，再逐步细化到各个功能模块，最后完成具体硬件实现。这种方法能够有效提高大型数字系统设计的可管理性和可扩展性。

8.3　数字系统设计的描述方法

数字系统设计的本质，是对硬件结构、逻辑功能以及系统运行行为进行描述。随着数字系统规模不断扩大，设计复杂度不断提高，传统依靠逻辑图直接搭建电

路的方法已经难以满足现代工程需求。现代数字系统设计更加注重抽象层次、模块划分和工程化开发，设计者通常借助硬件描述语言对系统进行建模、仿真和综合。

所谓“描述方法”，实际上是指设计者从不同角度表达数字系统的方法。有的方法强调系统整体结构，有的方法强调数据之间的逻辑关系，有的方法强调系统行为和时序特征。合理选择描述方法，不仅能够提高设计效率，还能够增强系统的可读性、可维护性和可移植性。因此，掌握数字系统设计中的常用描述方法，是学习现代数字系统设计的重要基础。

8.3.1 自顶向下设计方法

自顶向下（Top-Down）设计方法是现代数字系统设计中最常用、最典型的一种设计思想。该方法强调从系统整体功能出发，先确定系统总体结构，再逐步向下细化，最终完成各个底层模块的具体实现。

在实际设计过程中，设计者通常首先分析系统需要实现的总体功能，例如数据处理、状态控制、通信接口或显示控制等，然后根据功能需求将系统划分为若干相互独立的功能模块。每个模块再继续细分为更小的子模块，直到能够完成具体硬件实现为止。

例如，在设计一个数字时钟系统时，可以先将系统划分为时钟产生模块、计时模块、按键控制模块和显示模块；随后再进一步细化计时模块中的秒计数、分计数和时计数电路。这样，整个系统就形成了清晰的层次结构。

自顶向下设计方法具有明显的工程优势。由于系统结构在设计初期已经得到整体规划，因此有利于大型数字系统的组织和管理。不同设计人员可以分别负责不同模块，从而提高团队协作效率。同时，该方法还能够增强系统的可维护性和可扩展性，当某个模块需要升级时，通常不会影响整个系统结构。

在硬件描述语言设计中，自顶向下方法通常表现为模块嵌套和层次化调用结构。设计者通过顶层模块调用多个子模块，再由子模块调用更底层功能模块，从而逐步构建完整系统。现代 FPGA 和 ASIC 设计大多采用这种设计思想。

8.3.2 自底向上设计方法

与自顶向下方法不同，自底向上（Bottom-Up）设计方法强调先完成底层功能模块设计，再逐步组合形成完整系统。

在这种设计思想中，设计者首先完成一些基本功能单元，例如加法器、计数器、寄存器、译码器等模块，然后利用这些已有模块逐步构建更复杂的系统。其设计过程类似于“积木式”搭建。

自底向上设计方法适用于已经拥有成熟功能模块的情况。例如，在工程实践中，很多系统会直接调用已有的 IP 核，包括 FIFO、RAM 控制器、串口模块或通信接口模块等。这样能够减少重复设计工作，提高开发效率。

对于规模较小、结构相对固定的数字系统，自底向上方法也具有较好的适用性。由于底层模块通常已经经过验证，因此系统可靠性较高。

不过，对于大型复杂系统而言，如果仅采用自底向上方法，往往容易导致系统整体结构缺乏统一规划。因此，在现代数字系统开发中，更常采用自顶向下与自底向上相结合的方法：先利用自顶向下方法完成系统架构设计，再利用自底向上方法复用已有模块和 IP 资源。

8.3.3 结构化描述方法

结构化描述方法主要强调系统中各模块之间的连接关系，其核心思想是从硬件结构角度描述数字系统。

在结构化描述中，设计者更加关注模块之间如何连接、信号如何传递以及系统整体结构如何组织，而不是单纯描述逻辑功能。结构化描述方式与实际硬件连接关系较为接近，因此具有较强的直观性。

在硬件描述语言中，结构化描述通常通过模块例化实现。所谓模块例化，是指在一个模块内部调用其他模块，并完成端口连接。例如，一个复杂系统可能由若干加法器、寄存器和控制模块构成，设计者可以在顶层模块中将这些子模块连接起来，从而形成完整系统。

结构化描述方法有利于层次化设计和模块复用。由于各功能模块相对独立，因此当某个模块需要修改时，通常不会影响其他部分。同时，该方法也有利于团

队协同开发，不同设计人员可以分别完成不同模块设计。

在大型数字系统中，结构化描述常用于顶层系统设计。通过结构化方式，设计人员能够清晰地观察整个系统结构和信号流向，从而提高系统设计的规范性和可维护性。

8.3.4 数据流描述方法

数据流描述方法主要用于描述数据在系统中的传输和逻辑运算关系，其核心思想是利用逻辑表达式描述输入与输出之间的对应关系。

与结构化描述相比，数据流描述更加关注“数据如何变化”，而不是“硬件如何连接”。在硬件描述语言中，数据流描述通常采用连续赋值语句实现。

例如，对于一个简单与门：

$$Y = A * B$$

可以直接利用赋值语句描述输出与输入之间的逻辑关系。

数据流描述方式具有表达简洁、逻辑清晰的特点，特别适合组合逻辑电路设计。设计者无需显式描述门电路结构，只需描述逻辑关系即可，EDA 工具会自动完成逻辑综合。

在 RTL 级设计中，大量组合逻辑部分都采用数据流描述方式。例如译码器、编码器、多路选择器和算术逻辑运算单元等，都可以通过数据流方式高效实现。

此外，数据流描述还便于综合工具进行逻辑优化。EDA 软件能够根据逻辑关系自动优化门级结构，从而减少硬件资源消耗并提高系统性能。

8.3.5 行为描述方法

行为描述方法是从系统功能行为角度描述数字系统工作过程的一种方法。与结构化描述和数据流描述相比，行为描述不强调具体硬件结构，而更加关注系统“做什么”和“如何工作”。

在行为描述中，设计者通常采用过程语句、条件判断语句和循环语句描述系统行为。例如，一个计数器可以描述为“每当时钟到来时计数值加 1”，而不必关心具体由哪些触发器和逻辑门构成。

行为描述方法具有较高抽象层次，因此特别适合复杂控制逻辑和算法建模。

例如状态机控制、通信协议处理和数据运算流程等，通常都采用行为级描述。

由于行为描述更加接近软件程序的表达方式，因此代码可读性较强，设计效率较高。同时，行为描述还能够快速完成系统功能验证，有利于早期仿真分析。

不过，需要注意的是，并非所有行为级语句都能够被综合为实际硬件。在 RTL 设计中，必须遵循可综合设计规范，否则综合工具可能无法生成正确硬件电路。

8.3.6 RTL 级设计思想

RTL（Register Transfer Level）即寄存器传输级，是现代数字系统设计中最核心、最常用的抽象层次。

RTL 设计的本质，是描述数据在寄存器之间如何传输，以及这些数据如何在时钟控制下完成运算和状态更新。因此，RTL 级描述是一种面向寄存器与时钟行为的硬件描述方式，其核心是描述数据在寄存器之间的传输关系以及时序控制逻辑。

在 RTL 设计中，时钟信号具有核心作用。系统状态通常只在时钟边沿发生变化，因此 RTL 级设计能够较好地反映真实数字系统的时序行为。

例如，一个寄存器在时钟上升沿接收输入数据，可以描述为：

```
always @(posedge clk)
 q <= d;
```

这实际上描述了“数据在时钟控制下从输入寄存器传输到输出寄存器”的过程。

RTL 设计兼顾抽象性和可综合性。一方面，它不像门级描述那样复杂；另一方面，它又能够被综合工具直接转换为实际硬件。因此，RTL 级已经成为 FPGA 和 ASIC 设计的主流设计层次。

现代数字系统开发几乎都围绕 RTL 级展开，包括功能仿真、逻辑综合、时序分析和硬件实现等过程。

8.3.7 模块化设计思想

模块化设计是现代数字系统设计中的重要工程思想，其核心目标是将复杂系统划分为若干相对独立的功能模块。

通过模块划分，可以有效降低系统复杂度，使设计过程更加清晰和规范。例如，一个嵌入式数字系统可以划分为输入接口模块、数据处理模块、存储模块和显示模块等，每个模块分别完成特定功能。

模块化设计不仅有利于提高代码可读性，还能够增强系统的复用性。已经完成验证的模块可以在不同工程中重复使用，从而减少重复开发工作。

在团队开发过程中，模块化设计还能够提高协作效率。不同开发人员可以分别负责不同模块设计，最终通过统一接口完成系统集成。

此外，模块化设计还便于后期维护与升级。当系统需要扩展功能时，通常只需增加或修改部分模块，而不必重新设计整个系统。

在硬件描述语言中，每个模块通常对应一个独立功能单元。通过模块之间的层次调用，可以逐步构建复杂数字系统。

8.3.8 同步设计思想

同步设计是现代数字系统设计中的重要原则，其核心思想是利用统一时钟控制系统状态变化。

在同步系统中，大部分状态更新都发生在时钟边沿，因此系统运行具有明确节奏。这种统一时序管理方式能够显著提高系统稳定性和可靠性。

同步设计强调所有时序逻辑尽量工作在统一时钟域下，从而减少竞争冒险和时序异常问题。同时，同步设计还要求复位信号具有明确时序关系，以确保系统能够可靠初始化。

随着数字系统运行速度不断提高，时钟管理问题变得越来越重要。例如，在大型 FPGA 系统中，不同模块可能工作在不同频率下，因此需要合理划分时钟域，并采取同步处理措施。

此外，当异步信号进入同步系统时，还可能产生亚稳态问题。为了提高系统可靠性，通常需要采用双触发器同步等方法进行处理。

同步设计思想已经成为现代 FPGA 设计和 ASIC 设计的重要基础。规范的同步设计不仅能够提高系统稳定性，还能够简化后续时序分析和综合优化工作。

8.3.9 可综合设计规范

硬件描述语言既可以用于功能行为仿真，也可以用于硬件综合。但并不是所有语句都能够被综合工具转换为实际硬件，因此在数字系统设计中必须遵循可综合设计规范。

所谓“可综合”，是指描述语句能够被 EDA 工具正确转换为逻辑门、触发器和连线等实际硬件结构。如果设计代码不符合综合规范，虽然可能通过仿真，但无法生成正确硬件电路。

在 RTL 设计过程中，设计人员通常需要合理划分组合逻辑与时序逻辑，并规范使用时钟触发语句。例如，时序逻辑一般采用时钟边沿触发，而组合逻辑则需要保证所有输出信号在各种条件下均有明确赋值。

此外，在 Verilog 设计中，还需要正确区分阻塞赋值与非阻塞赋值。一般情况下，组合逻辑常采用阻塞赋值，而时序逻辑常采用非阻塞赋值。这样能够更符合真实硬件运行规律。

规范化设计不仅能够提高综合质量，还能够减少竞争冒险、锁存器误生成以及时序错误等问题。同时，良好的设计规范也有利于提高代码可读性和后期维护效率。

随着数字系统规模不断扩大，可综合设计规范已经成为现代 FPGA 与 ASIC 开发中的重要工程要求。

8.4 用可编程逻辑器件进行数字系统设计

随着集成电路技术的发展，可编程逻辑器件已经成为数字系统实现的重要平台。设计者可以利用 Verilog HDL 完成数字系统建模，并借助 EDA 工具实现逻辑综合、布局布线以及硬件下载。

可编程逻辑器件主要包括 CPLD 与 FPGA。其中，FPGA 具有逻辑资源丰富、可重复编程以及开发周期短等优势，广泛应用于数字系统开发。

8.4.1 逻辑门电路的 Verilog 描述

逻辑门是数字系统的基本组成单元。在 Verilog HDL 中，可以采用多种方式

实现逻辑门描述，包括：

1. 门级建模；
2. 数据流建模；
3. 行为级建模。

（1）与门描述

```
module and_gate(
    input a,
    input b,
    output y
);
assign y = a & b;
endmodule
```

上述代码采用连续赋值语句实现与门逻辑。

（2）或门描述

```
module or_gate(
    input a,
    input b,
    output y
);

assign y = a | b;
endmodule
```

（3）非门描述

```
module not_gate(
    input a,
    output y
);
assign y = ~a;
endmodule
```

（4）异或门描述

```
module xor_gate(
    input a,
    input b,
    output y
);
assign y = a ^ b;
endmodule
```

（5）门级原语描述

Verilog 还支持直接调用门级原语。

```
module and_gate2(
    input a,
    input b,
    output y
);
and u1(y, a, b);
endmodule
```

门级建模更接近底层硬件结构，但在复杂系统设计中通常采用 RTL 级描述方式。

8.4.2 组合逻辑电路的 Verilog 描述

组合逻辑电路的输出仅与当前输入有关，因此在 Verilog 设计中通常采用连续赋值语句或组合过程块实现。

（1）二选一数据选择器

```
module mux2_1(
    input a,
    input b,
    input sel,
    output y
);
assign y = sel ? b : a;
endmodule
```

（2）四选一数据选择器

```
module mux4_1(
    input [3:0] d,
    input [1:0] sel,
    output reg y
);
always @(*) begin
    case(sel)
        2'b00: y = d[0];
        2'b01: y = d[1];
        2'b10: y = d[2];
        2'b11: y = d[3];
    endcase
```

```
end
endmodule
```

（3）编码器设计

```
module encoder4_2(
    input [3:0] d,
    output reg [1:0] y
);
always @(*) begin
    case(d)
        4'b0001: y = 2'b00;
        4'b0010: y = 2'b01;
        4'b0100: y = 2'b10;
        4'b1000: y = 2'b11;
        default: y = 2'b00;
    endcase
end
endmodule
```

（4）译码器设计

```
module decoder2_4(
    input [1:0] a,
    output reg [3:0] y
);
always @(*) begin
    case(a)
        2'b00: y = 4'b0001;
        2'b01: y = 4'b0010;
        2'b10: y = 4'b0100;
        2'b11: y = 4'b1000;
    endcase
end
endmodule
```

（5）比较器设计

```
module comparator(
    input [3:0] a,
    input [3:0] b,
    output reg greater,
    output reg equal,
    output reg less
);
always @(*) begin
```

```
    if(a > b) begin
        greater = 1'b1;
        equal = 1'b0;
        less = 1'b0;
    end
    else if(a == b) begin
        greater = 1'b0;
        equal = 1'b1;
        less = 1'b0;
    end
    else begin
        greater = 1'b0;
        equal = 1'b0;
        less = 1'b1;
    end
end
endmodule
```

8.4.3 触发器的 Verilog 描述

触发器是时序逻辑系统中的基本存储单元，其状态变化通常由时钟边沿触发。在 Verilog 中，触发器一般采用时序过程块描述。

（1）D 触发器

```
module dff(
    input clk,
    input rst_n,
    input d,
    output reg q
);
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        q <= 1'b0;
    else
        q <= d;
end
endmodule
```

（2）带使能控制的 D 触发器

```
module dff_en(
    input clk,
    input rst_n,
```

```
    input en,
    input d,
    output reg q
);
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        q <= 1'b0;
    else if(en)
        q <= d;
end
endmodule
```

（3）JK 触发器

```
module jk_ff(
    input clk,
    input rst_n,
    input j,
    input k,
    output reg q
);
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        q <= 1'b0;
    else begin
        case({j,k})
            2'b00: q <= q;
            2'b01: q <= 1'b0;
            2'b10: q <= 1'b1;
            2'b11: q <= ~q;
        endcase
    end
end
endmodule
```

（4）T 触发器

```
module t_ff(
    input clk,
    input rst_n,
    input t,
    output reg q
);
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
```

```
        q <= 1'b0;
    else if(t)
        q <= ~q;
end
endmodule
```

8.4.4 时序逻辑电路的 Verilog 描述

时序逻辑电路不仅与当前输入有关，还与历史状态有关，因此通常需要状态存储单元。

（1）计数器设计

```
module counter4(
    input clk,
    input rst_n,
    output reg [3:0] cnt
);
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        cnt <= 4'b0000;
    else
        cnt <= cnt + 1'b1;
end
endmodule
```

（2）可控计数器设计

```
module counter_enable(
    input clk,
    input rst_n,
    input en,
    output reg [3:0] cnt
);
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        cnt <= 4'b0000;
    else if(en)
        cnt <= cnt + 1'b1;
end
endmodule
```

（3）移位寄存器设计

```
module shift_reg(
    input clk,
    input rst_n,
    input d,
    output reg [3:0] q
);
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        q <= 4'b0000;
    else
        q <= {q[2:0], d};
end
endmodule
```

8.4.5 复杂数字电路的 Verilog 描述

随着系统规模扩大，单一模块已经难以满足复杂功能需求。因此，需要采用层次化、模块化与参数化设计思想完成复杂数字系统开发。

（1）层次化设计

随着数字系统规模不断扩大，一个完整系统往往难以仅用单一模块实现，因此通常需要将系统划分为多个功能相对独立的子模块，再通过顶层模块进行统一连接与管理。这种设计思想称为层次化设计。

在 Verilog 中，层次化设计通常通过“模块例化”实现，即在一个模块内部调用其他功能模块，并完成端口连接。

```
module top_module(
    input a,
    input b,
    input sel,
    output y
);
wire w1;
and_gate u1(
    .a(a),
    .b(b),
    .y(w1)
);
mux2_1 u2(
    .a(w1),
```

```
    .b(a),
    .sel(sel),
    .y(y)
);
endmodule
```

上述代码中：and_gate 模块首先完成与逻辑运算，运算结果通过中间信号 w1 输出，mux2_1 模块再根据选择信号 sel 决定最终输出。其中：u1 和 u2 为模块实例名，wire w1 用于连接不同模块之间的信号。这种设计方式类似于“搭积木”，设计者可以先完成底层功能模块设计，再逐步构建更复杂系统。

（2）参数化设计

在数字系统开发中，很多功能模块结构相似，仅数据位宽或规模不同。如果针对不同位宽分别重新编写代码，会导致重复开发，降低设计效率。因此，Verilog 提供了参数（parameter）机制，用于实现参数化设计。

```
module counter #(
    parameter WIDTH = 8
)(
    input clk,
    input rst_n,
    output reg [WIDTH-1:0] cnt
);
always @(posedge clk or negedge rst_n) begin
    if(!rst_n)
        cnt <= 0;
    else
        cnt <= cnt + 1'b1;
end
endmodule
```

上述代码中：WIDTH 为参数，默认值为 8，cnt 的位宽由参数决定。 因此：当 WIDTH=8 时，为 8 位计数器，当 WIDTH=16 时，则自动变为 16 位计数器。 例如：

```
counter #(8) u1(...);
counter #(16) u2(...);
```

分别表示：8 位计数器和 16 位计数器。 参数化设计能够显著提高模块复用能力，使同一模块适用于不同应用场景。

（3）存储器设计

存储器用于保存数据和系统状态，是数字系统中的重要组成部分。在 Verilog 中，可以利用数组形式描述 RAM 存储结构。

```
module ram(
    input clk,
    input we,
    input [3:0] addr,
    input [7:0] din,
    output reg [7:0] dout
);
reg [7:0] mem [15:0];
always @(posedge clk) begin
    if(we)
        mem[addr] <= din;
    dout <= mem[addr];
end
endmodule
```

上述代码中：“reg [7:0] mem [15:0]”表示：存储器共有 16 个单元，每个单元宽度为 8 位。

其中：addr 为地址输入，din 为写入数据，dout 为输出数据；we 为写使能信号。工作过程如下：

当 we=1 时，在时钟上升沿将 din 写入指定地址，同时从当前地址读取数据输出到 dout。该 RAM 属于同步 RAM，其读写操作均在时钟控制下完成。这种描述方式能够被综合工具自动识别，并映射为 FPGA 内部的块 RAM 或分布式 RAM 资源。随着系统规模不断扩大，片上存储资源已经成为 FPGA 设计中的重要组成部分。

（4）复杂数字系统设计流程

复杂数字系统开发通常包括以下步骤：

1) 系统需求分析；
2) 系统架构划分；
3) RTL 代码设计；
4) 仿真验证；
5) 逻辑综合；
6) 时序优化；

7) 下载调试；

8) 系统测试。

（5）Verilog 设计中的工程化思想

随着数字系统规模不断扩大，现代 Verilog 设计已经不再只是完成简单逻辑功能，而逐渐发展为一种工程化开发过程。尤其在大型 FPGA 与 ASIC 项目中，系统复杂度高、参与人员多，因此工程化思想已成为数字系统设计的重要基础。

早期数字电路设计更加关注“功能能否实现”，而现代数字系统不仅要求功能正确，还要求具有良好的可读性、可维护性、可扩展性和可复用性。因此，Verilog 设计越来越强调规范化开发。

工程化设计首先重视代码规范。统一的命名方式、缩进格式和注释风格，能够提高代码可读性，方便团队成员之间协同开发，同时也有利于后续调试与维护。

在大型工程中，团队协作十分重要。不同设计人员通常分别负责不同功能模块，因此模块接口定义和设计文档必须统一，否则容易导致系统集成困难。

随着系统规模增大，版本管理也变得越来越重要。现代数字系统开发通常采用 Git 等工具对代码进行统一管理，从而避免代码混乱和功能丢失问题。

此外，现代 Verilog 设计还强调自动化测试与模块复用。通过测试平台自动完成验证，可以提高测试效率；通过复用成熟 IP 核，则能够缩短开发周期并提高系统可靠性。

因此，现代数字系统设计已经逐渐从传统“电路设计”发展为“工程化系统开发”。设计人员不仅需要掌握硬件描述语言，还需要具备系统设计、团队协作和工程管理等综合能力。

本章小结

本章系统介绍了数字系统设计基础及基于 Verilog HDL 的数字系统开发方法。

首先，介绍了数字系统的基本概念、组成结构、主要特点以及现代数字系统的发展趋势，说明了硬件描述语言在数字系统设计中的重要作用。

随后，分析了数字系统设计的一般过程，包括需求分析、系统建模、功能仿真、逻辑综合、布局布线以及硬件测试等内容，体现了现代 EDA 设计流程的完整性。

接着，介绍了数字系统的主要描述方法，包括结构化描述、数据流描述、行为描述以及 RTL 级设计思想，强调了模块化与同步化设计原则。